E1CSA313 DESIGN PROJECT-1 · ALLIANCE UNIVERSITY

Chain of Record: Blockchain Architecture and Files

What the blockchain does in our application, the architecture around it, the steps it takes, and every blockchain file with its purpose.

Contents

1. What the blockchain does here
2. Architecture
3. Inside the smart contract
4. How it works, step by step (with the file that does each step)
5. The hashes that connect everything
6. Blockchain files and their purpose
7. Live deployment

Team: Manu M P · Tarun Kumar S · Anilkumar U K · Pruthviraj D N
Mentor: Dr. Saravanan P

Blockchain: Ethereum Sepolia · Smart contract: `CredentialRegistry` at `0x2E36EB891D77c7f9dE0BC1b00D17ECb00f37D15E`. Companion document: *Blockchain_In_Our_Application.pdf* (live block data and verification walkthrough).

1. What the blockchain does here

The blockchain is the **single source of truth** for whether a certificate is genuine. It is Ethereum (public test network Sepolia), and our code on it is one smart contract, `CredentialRegistry`. It has four jobs:

1. Decide who may issue
The platform owner authorises each university's wallet address (`setIssuer`). Any other address is refused by the contract.

2. Record certificates
A university records a certificate's fingerprint (its **docHash**), alone or many at once under one **Merkle root**.

3. Record revocations
Only the university that issued a certificate can revoke it. The revocation is permanent and timestamped.

4. Answer verifications
Anyone can ask, for free: is this exact hash recorded, by an authorised university, and is it revoked?

What the blockchain does *not* do

Not on the blockchain	Where it is instead	Why
Names, marks, dates of birth, any personal data	PostgreSQL (Supabase)	A public chain cannot be erased; personal data must not be on it
The certificate document itself	IPFS, encrypted (AES-256-GCM)	Storing documents on Ethereum is expensive; the chain only needs the hash
Reading the certificate photo (OCR)	The verifier's browser	Computation, not record-keeping
Revocation reason text	PostgreSQL; only `keccak256(reason)` is on-chain	Proves the reason later without publishing it

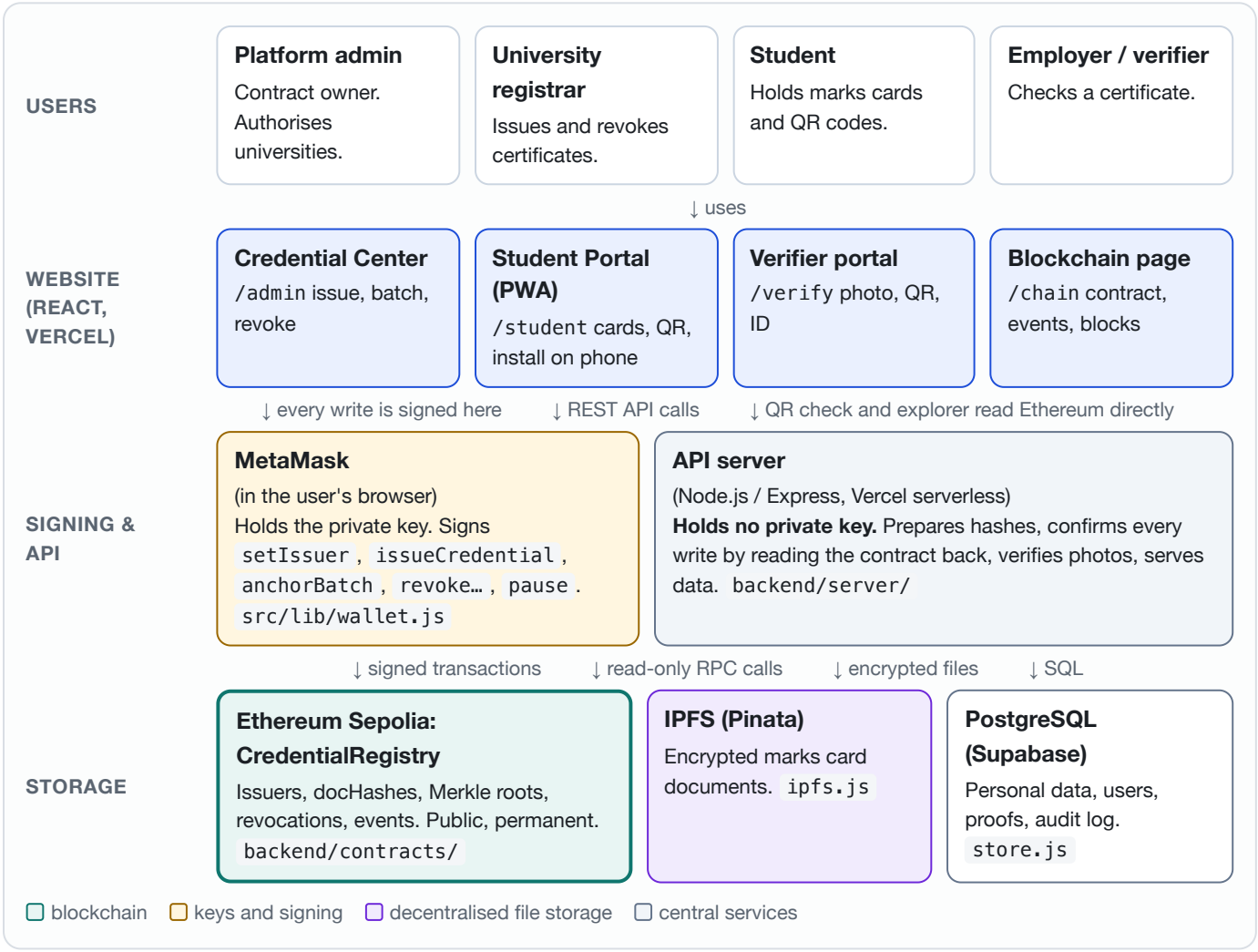
The core idea in one line. Anything about a certificate can be recomputed into a 32-byte hash. If that hash is recorded in the contract by an authorised university and not revoked, the certificate is genuine. If even one mark is changed, the hash changes and the contract no longer has it.

Why Ethereum, and why a smart contract

Property	What it gives us
Thousands of independent nodes	No single party (including us) can change or delete a record.
Blocks chained by parent hash	Changing an old record would break every block after it.
Signed transactions	Every record proves which university's key created it.
Smart contract rules	The rules (who may issue, who may revoke, no duplicates) are enforced by every node, not by our server.
Free public reads	Employers can verify without an account and without our server (QR check).

2. Architecture

Five parts. The blockchain sits at the bottom as the trust anchor; everything above it either writes to it (through MetaMask) or reads from it.



Who talks to the blockchain, and how

Component	Access	How
MetaMask	WRITE	The only way anything is written. Signs with the user's own key and broadcasts to Sepolia.
API server	READ	Through a JSON-RPC provider with no signer (`runtime.js → connectContract`). Confirms writes, verifies, reads history.
Browser QR check & explorer	READ	Directly through a public RPC (`ethereum-sepolia-rpc.publicnode.com`), bypassing our server.
Deploy and import scripts	WRITE	Operator tools on the team's machine: deploy the contract, and load the test dataset with test keys.
Automated tests	LOCAL	Hardhat's in-process Ethereum; test signers play the role of MetaMask.

Trust boundaries

If this part is compromised…	…this is what happens
API server	It cannot issue or revoke (no keys). It could lie in its own answers, but the QR check and Etherscan still show the truth.

Database	An edited record no longer hashes to the on-chain docHash → INVALID.
One university's key	The owner suspends it with `setIssuer(…, false)`; its certificates then verify as untrusted. `pause()` stops all writes in an emergency.
IPFS gateway	A changed file fails decryption (GCM tag) or the hash check. Another gateway can be used.

3. Inside the smart contract

`backend/contracts/CredentialRegistry.sol`: 226 lines, Solidity 0.8.24, built on audited OpenZeppelin parts: `Ownable2Step` (ownership with a two-step transfer), `Pausable` (emergency stop) and `MerkleProof` (batch proofs).

What it stores

Storage	Holds
`issuers[address] → {active, name}`	Which wallet addresses may issue, and the university's name
`credentials[key] → Credential`	For each individually issued certificate: `docHash`, `issuer`, `issuedAt`, `revokedAt`, `status` (None / Valid / Revoked). `key = keccak256(credentialId)`
`keyByHash[docHash] → key`	Find a certificate from its hash alone; also blocks registering the same hash twice
`batches[batchId] → Batch`	For each batch: Merkle `root`, `issuer`, `anchoredAt`, `count`
`rootUsed[root]`	Blocks anchoring the same root twice
`batchLeafRevokedAt[leaf]`	Revocation time of a certificate inside a batch (0 = not revoked)
`totalIssued`, `totalRevoked`, `batchCount`	Counters shown on the Blockchain page

Functions

Function	Guard	Effect
`setIssuer(address, name, active)`	`onlyOwner`	Authorise or suspend a university. Emits `IssuerUpdated`.
`pause()` / `unpause()`	`onlyOwner`	Stop or resume all issuing and revoking. Reads keep working.
`issueCredential(id, docHash)`	`onlyIssuer`, not paused	Record one certificate. Emits `CredentialIssued`.
`revokeCredential(id, reasonHash)`	original issuer, not paused	Mark it revoked. Emits `CredentialRevoked`.
`anchorBatch(root, count)`	`onlyIssuer`, not paused	Record a Merkle root for `count` certificates. Emits `BatchAnchored`.
`revokeBatchCredential(batchId, id, docHash, proof, reasonHash)`	batch's issuer, valid proof	Revoke one certificate inside a batch. Emits `CredentialRevoked`.
`getCredential(id)`, `getCredentialByHash(docHash)`	anyone (free read)	Return the record and whether its issuer is still active
`verifyCredential(id, docHash)`	anyone (free read)	Does this ID hold exactly this hash?

`verifyBatchCredential(batchId, id, docHash, proof)`	anyone (free read)	Is the certificate in the batch (Merkle proof), issuer, issuer active, revoked time
`leafOf(id, docHash)`	pure	The Merkle leaf formula, so off-chain code can match it exactly

Rules it enforces (custom errors)

Error	When
`NotAuthorizedIssuer`	Sender is not an active issuer
`CredentialAlreadyExists` / `HashAlreadyRegistered`	The ID or the hash is already recorded
`NotOriginalIssuer`	A university tries to revoke another university's certificate
`AlreadyRevoked`, `CredentialNotFound`, `BatchNotFound`	Revoking twice, or something that does not exist
`RootAlreadyAnchored`, `InvalidProof`, `EmptyValue`	Duplicate batch, wrong Merkle proof, zero hash or empty value

A transaction that hits any of these reverts: nothing is stored, and MetaMask shows the error name (`wallet.js` turns it into a readable message).

4. How it works, step by step

A. Setting up (once)

1	**Deploy**	`npm run deploy:sepolia` → backend/scripts/deploy.js compiles the contract with Hardhat, deploys it, and writes the address, ABI and deploy block to backend/deployments/sepolia.json. Both the server and the website read that file.
2	**Register**	A university applies at `/register/institution` with its MetaMask address (institutions.js). It stays *pending*.
3	**Authorise**	The platform admin clicks Approve → MetaMask signs `setIssuer(address, name, true)` → the server checks `issuers(address).active` on-chain, then marks it approved.

B. Issuing one certificate

1	**Prepare**	Registrar fills the form → `POST /api/credentials/prepare` → server builds the document string and its SHA-256 **docHash** (hash.js, certificateView.js).
2	**Sign**	wallet.js `sendTransaction('issueCredential', [id, docHash], registrarAddress)`: connects MetaMask, switches to Sepolia, checks the right account, and asks the user to sign.
3	**Mine**	Sepolia validators include the transaction in a block. Every node runs the contract: `onlyIssuer`, not paused, new ID, new hash.
4	**Confirm**	Browser sends the transaction hash → `POST /api/credentials` → chain.js `transaction(txHash)` + `getById(id)`: succeeded, sent to our contract, this ID holds this docHash, signed by this university. Only then is the record saved (app.js).
5	**Store**	Record saved to PostgreSQL (store.js); the document is encrypted and pinned to IPFS (ipfs.js); the marks card with its QR is drawn (certificateSvg.js).

C. Issuing many at once (Merkle batch)

1	**Build tree**	`POST /api/credentials/batch/prepare` → each certificate's leaf, then the root (merkle.js, same formula as the contract's `leafOf`).
2	**Sign**	MetaMask signs `anchorBatch(root, count)` : one transaction for the whole batch.
3	**Confirm**	`POST /api/credentials/batch` → server reads `batches(batchId)` : same root, same count, same issuer → saves each certificate with its own Merkle proof.

D. Revoking

1	**Sign**	Registrar enters a reason → MetaMask signs `revokeCredential(id, keccak256(reason))` (or the batch version with the proof).
2	**Confirm**	`POST /api/credentials/:id/revoke` → server checks the contract now says revoked, by the original issuer, with the same reason hash → saves the reason text.

E. Verifying

1	**Photo**	PhotoVerifyTab.jsx + photoEvidence.js read the photo in the browser (OCR + QR) → `POST /api/verify/photo` → photoVerify.js: on chain? issuer authorised? details match? record hashes to the on-chain docHash? QR hash matches? revoked?
2	**QR**	DirectChainCheck.jsx + chainVerify.js `verifyFromQr` : IPFS → decrypt → SHA-256 → contract → block. **No server involved.**
3	**ID / hash**	`GET /api/verify/:id` or `POST /api/verify/hash` → app.js `verifyById` / `verifyByHash` → `evaluate()` .
4	**Proof**	Every VALID / REVOKED answer carries: network, contract, transaction, block number, block hash, confirmations, batch, Merkle root, proof valid.

F. Watching the chain

ChainExplorer.jsx (`/chain`) uses chainVerify.js `contractOverview()` and `blockDetails()` to show the contract's counters, the latest `IssuerUpdated` / `CredentialIssued` / `BatchAnchored` / `CredentialRevoked` events, and each block's hash and parent hash, read straight from Ethereum.

5. The hashes that connect everything

```
document   = canonical line  + "\n" + one line per semester
canonical  = cert_no|reg_no|NAME|dob|program|score|year|univ
docHash    = SHA-256(document)                                  hash.js, certificateView.js
credential key  = keccak256(credentialId)                       contract
leaf       = keccak256( keccak256( abi.encode(keccak256(id), docHash) ) )   contract leafOf, merkle.js
root       = pairwise keccak256 of sorted leaves, up to one hash  merkle.js, OpenZeppelin MerkleProof
reasonHash = keccak256(reason text)                             wallet.js reasonHash
QR link    = …/verify/<cert no>/<docHash>#cid=<IPFS CID>&k=<AES key>   certificateView.js verifyUrl
```

The server (Node) and the browser (Web Crypto) compute the same docHash from the same function in `src/lib/certificateView.js` , so a photo check, a QR check and the contract all agree.

6. Blockchain files and their purpose

Access: `WRITE` sends transactions · `READ` reads the chain · `LOCAL` local test chain only · `NONE` no chain access, but prepares or uses chain data.

Smart contract and its tooling

File	Lines	Access	Purpose and what it does
`backend/contracts/CredentialRegistry.sol`	226	ON-CHAIN	**The blockchain logic.** Stores issuers, certificates, batches and revocations; enforces every rule (section 3).
`backend/hardhat.config.cjs`	23	CONFIG	Compiler 0.8.24 with optimizer; networks: in-process `hardhat`, `localhost` (port 8545), `sepolia` (from `.env`).
`backend/deployments/sepolia.json`	859	CONFIG	The live contract's address, ABI, chain id, owner and deploy block. Read by the server and bundled into the website.
`backend/deployments/localhost.json`	859	CONFIG	The same for a local Hardhat chain during development.
`backend/scripts/deploy.js`	42	WRITE	Deploys the contract and writes `deployments/<network>.json`.

API server (backend/server)

File	Lines	Access	Purpose and what it does
`chain.js`	275	READ	**The blockchain bridge**: the only server module that talks to the contract. Reads credentials, batches, issuers, transactions, events and stats; 8 s timeout; RPC failure → `ChainUnavailableError` (503, never a verdict). Its write helpers are used only by tests with a signer.
`runtime.js`	51	READ	Connects to the contract **without a private key** (`connectContract`) and wires chain, database, IPFS and auth into the app.
`app.js`	897	READ	REST API: *prepare* → *confirm* for issue, batch, revoke and admin actions (checks the contract before saving); verify by ID / hash / photo; the proof (block, block hash, confirmations, Merkle proof).
`photoVerify.js`	270	READ	Photo verdict: on chain → issuer → field comparison → docHash vs contract → QR hash → revocation.
`hash.js`	10	NONE	Computes the docHash (SHA-256) exactly as the browser does.
`merkle.js`	48	NONE	Builds Merkle trees and proofs, matching the contract's `leafOf` and OpenZeppelin's `MerkleProof`.
`ipfs.js`	56	NONE	Encrypts a document (AES-256-GCM) and pins it to IPFS through Pinata; returns the CID and key.
`institutions.js`	89	READ	University lifecycle: register with a wallet address; approve or suspend only after `issuers(address)` on-chain agrees.
`config.js`	117	CONFIG	Network (`localhost` or `sepolia`), RPC URL, Pinata; warns if an old private-key variable is still set.
`index.js`, `/api/index.js`	34 / 37	CONFIG	Start the API locally, or as a Vercel serverless function, with `deployments/<network>.json`.

Website (src)

File	Lines	Access	Purpose and what it does

`lib/wallet.js`	98	WRITE	**Every blockchain write happens here, through MetaMask**: connect, switch or add Sepolia, check the account, sign, wait for the block; readable errors; `reasonHash`; Etherscan links.
`lib/chainVerify.js`	140	READ	Serverless checking: `verifyFromQr` (IPFS → decrypt → hash → contract → block), `contractOverview`, `blockDetails`.
`lib/certificateView.js`	119	NONE	The exact text that is hashed (canonical line and marks), credential IDs, QR payload and link.
`lib/certificateSvg.js`	195	NONE	Draws the marks card with its QR code and the Ethereum transaction or block.
`lib/qr.js`	42	NONE	Reads a scanned QR string: certificate number, docHash, IPFS address and key.
`lib/httpApi.js`	207	WRITE	API client: for every write, asks the server to prepare, signs through `wallet.js`, then asks the server to confirm.
`pages/ChainExplorer.jsx`	172	READ	The Blockchain page: contract status, counters, events, block hash and parent hash.
`components/DirectChainCheck.jsx`	95	READ	Shows the QR check's steps and verdict, without the server.
`components/PhotoVerifyTab.jsx`	223	NONE	Photo upload and result screen with the on-chain proof.
`components/WalletAppCard.jsx`	59	NONE	Student wallet: install the portal as a phone app (PWA).

Scripts

File	Lines	Access	Purpose and what it does
`scripts/import-karnataka.js`	445	WRITE	Loads the dataset: authorises 17 universities, funds their test wallets, each signs one Merkle batch, then revokes 86. Resumable; reuses a batch already on-chain.
`scripts/lib/dataset-chain.js`	83	LOCAL	Builds the same dataset on Hardhat's in-process chain for evaluations.
`scripts/evaluate-verification.js`	89	LOCAL	Runs 666 labelled verification requests against the contract (666/666).
`scripts/make-sample-photos.js`	180	LOCAL	Renders and degrades marks card photos, then runs the photo check against the contract.

Tests (backend/test, 92 passing)

File	Lines	Access	What it proves
`CredentialRegistry.test.js`	126	LOCAL	Contract issue, revoke, issuer and pause rules
`contract-reverts.test.js`	77	LOCAL	Every revert path not covered elsewhere
`batch.test.js`	171	LOCAL	Merkle batches: anchor, prove, revoke one, reject a forged proof
`scenarios.test.js`	190	LOCAL	End-to-end API scenarios through MetaMask-style signing; fail-safe 503 when the chain is down
`institutions.test.js`	210	LOCAL	Each university signs with its own wallet; no cross-university revokes
`photo.test.js`	194	LOCAL	Photo → VALID / INVALID / REVOKED against the contract

`security.test.js`	282	LOCAL	Auth, rate limits, headers, pause, two-step ownership, server holds no keys
`helpers.js`	117	LOCAL	Deploys a fresh contract per test; signers stand in for MetaMask

Retired: the team's earlier self-built blockchain (3 Proof-of-Authority nodes) is kept in `archive/own-blockchain/` and is not used by the running system.

7. Live deployment

Item	Value
Network	Ethereum Sepolia, chain id 11155111
Contract	`0x2E36EB891D77c7f9dE0BC1b00D17ECb00f37D15E`
Deploy block / transaction	12,054,016 · `0x2ac18effe71f220a88327f4bcf2634f2d2addc47f1fab54703e60603a003e5fe`
Owner (platform)	`0xD5E2E1123700Af43EDbAa94dc1b9e30EaE8ae231`
Authorised universities	17, each with its own wallet
Certificates	1,604 in 17 Merkle batches (blocks 12,054,064 to 12,054,347)
Revocations	86, in 23 blocks from 12,054,350
Check it	`sepolia.etherscan.io/address/0x2E36EB891D77c7f9dE0BC1b00D17ECb00f37D15E`

Limits. Sepolia is Ethereum's test network (real blocks and consensus, test ETH). The API server and database are central for personal data, sign-in and photo checks; the contract, IPFS and the QR check do not depend on them. The student wallet is a PWA, not Flutter. The dataset is synthetic.